

Guia de Deploy Docker na VPS

Instalação, segurança, backup e operação

Sistema Clínico MVP

VERSÃO	1.0
EMISSÃO	13 de agosto de 2026
CLASSIFICAÇÃO	Uso interno
STATUS	Guia operacional

CONDIÇÃO DE VIGÊNCIA

Use credenciais próprias do ambiente e mantenha a chave privada age fora da VPS. Teste backup e restauração em ambiente isolado antes da produção.

Este guia mostra como subir o Sistema Clinico MVP em uma VPS usando Docker Compose.

Arquitetura

O deploy usa cinco containers:

- caddy: proxy reverso publico com HTTPS automatico.
- frontend: aplicacao Nuxt acessivel publicamente apenas pelo Caddy.
- backend: API Flask rodando com Gunicorn, acessivel apenas pela rede interna Docker.
- mysql: banco MySQL persistido em volume Docker.
- redis: cache Redis persistido em volume Docker.

O Firebird/SPDATA fica fora do Docker e deve estar acessivel pela VPS via rede.

Arquivos adicionados

- docker-compose.yml: orquestra todos os servicos.
- Caddyfile: configura o proxy reverso HTTPS para o frontend.
- .env.example: modelo das variaveis de producao.
- backend/Dockerfile: imagem do Flask/Gunicorn.
- backend/.dockerignore: evita copiar arquivos locais para a imagem.
- frontend/Dockerfile: imagem do Nuxt em producao.
- frontend/.dockerignore: evita copiar node_modules, builds locais e .env.

Observacao: o container backend executa flask db upgrade automaticamente antes de iniciar o Gunicorn. Assim, novas migrations sao aplicadas no start do backend durante o deploy.

Antes de publicar

1. Remova credenciais reais do repositorio.
2. Nao versione arquivos .env.
3. Troque senhas que ja tenham sido compartilhadas ou commitadas.
4. Confirme que a VPS consegue acessar o servidor Firebird/SPDATA.
5. Confirme que as portas 80 e 443 da VPS estao liberadas.
6. Aponte o dominio/subdominio para o IP publico da VPS antes de subir o Caddy.

Importante: existe um arquivo local chamado VPS - Acesso.md com credenciais. Remova esse arquivo do repositorio e troque a senha da VPS antes de usar em producao.

Instalar Docker na VPS

Os comandos abaixo consideram uma VPS Ubuntu/Debian com acesso root.

```
apt update
apt install -y ca-certificates curl gnupg
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" >
/etc/apt/sources.list.d/docker.list
apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
docker --version
docker compose version
```

Se a VPS for Debian e o repositório acima falhar, use o instalador oficial simplificado:

```
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

Enviar o projeto para a VPS

Use Git ou copie o projeto para a VPS. Exemplo usando Git:

```
cd /opt
git clone URL_DO_SEU_REPOSITORIO sistema-clinico-mvp
cd sistema-clinico-mvp
```

Se o projeto já existir na VPS:

```
cd /opt/sistema-clinico-mvp
git pull
```

Configurar variáveis de ambiente

Crie o arquivo .env na raiz do projeto a partir do exemplo:

```
cp .env.example .env
nano .env
```

Preencha as variáveis:

```
APP_DOMAIN=sistema.seudominio.com.br
Nuxt_ENABLE MOCK_AUTH=false
Nuxt_AUTH_COOKIE_SECURE=true
TZ=America/Sao_Paulo
GUNICORN_WORKERS=3
GUNICORN_TIMEOUT=120

MYSQL_DATABASE=sistema_clinico_mvp
MYSQL_USER=clinico
MYSQL_PASSWORD=senha_forte_do_mysql
MYSQL_ROOT_PASSWORD=senha_forte_do_root_mysql

SECRET_KEY=senha_forte_para_flask
JWT_SECRET_KEY=senha_forte_para_jwt

FIREBIRD_HOST=ip_ou_host_do_firebird
FIREBIRD_PORT=3050
FIREBIRD_DATABASE=/caminho/para/o/banco.fdb
FIREBIRD_USER=usuario_firebird
FIREBIRD_PASSWORD=senha_firebird
FIREBIRD_CHARSET=WIN1252
```

Observacoes:

- MYSQL_DATABASE, MYSQL_USER e MYSQL_PASSWORD sao usados pelo Flask via SQLAlchemy_DATABASE_URI.
- Nuxt_FLASK_BASE_URL ja e definido no docker-compose.yml como http://backend:5000.
- Nuxt_ENABLE MOCK_AUTH=false impede login mockado em producao.
- Nuxt_AUTH_COOKIE_SECURE=true faz o cookie de autenticao funcionar apenas em HTTPS.
- APP_DOMAIN deve ser somente o dominio/subdominio, sem http:// ou https://.
- TZ=America/Sao_Paulo mantem backend, frontend e MySQL no fuso esperado.
- GUNICORN_WORKERS e GUNICORN_TIMEOUT controlam o Gunicorn do backend sem rebuild da imagem.
- FIREBIRD_HOST nao pode ser localhost, porque dentro do container localhost aponta para o proprio container.

Subir a aplicacao

Antes de subir, confirme que o DNS do dominio ja aponta para a VPS e que as portas 80 e 443 estao liberadas.

Na raiz do projeto, execute:

```
docker compose up -d --build
```

Durante esse processo, o backend aplica as migrations do Flask automaticamente antes de iniciar a API.

Confira se os containers subiram:

```
docker compose ps
```

O serviço caddy deve aparecer como Up. Os serviços mysql, redis, backend e frontend devem aparecer como healthy após alguns segundos.

Se o backend não ficar healthy, verifique os logs. Uma falha de migration impede o Gunicorn de iniciar.

Veja os logs se precisar diagnosticar:

```
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f caddy
docker compose logs -f mysql
```

Depois disso, acesse:

```
https://APP_DOMAIN
```

Exemplo:

```
https://sistema.seudominio.com.br
```

Liberar firewall

Se estiver usando ufw:

```
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 443/udp
ufw enable
ufw status
```

O frontend, backend, MySQL e Redis não precisam ser expostos publicamente. O acesso público fica apenas no Caddy.

Atualizar o deploy

Quando houver novas alterações no repositório:

```
cd /opt/sistema-clinico-mvp
git pull
docker compose up -d --build
docker image prune -f
```

As migrations também são executadas automaticamente nesse fluxo, porque o container backend roda flask db upgrade a cada start.

Parar e reiniciar

Parar todos os containers:

```
docker compose down
```

Reiniciar:

```
docker compose up -d
```

Reiniciar apenas um serviço:

```
docker compose restart caddy
docker compose restart backend
docker compose restart frontend
```

Backup do MySQL

Os backups devem ser compactados e criptografados com age. A VPS guarda apenas o destinatário público; a identidade privada deve permanecer fora do servidor, sob custódia institucional.

Instale o age no host da VPS:

```
apt update
apt install -y age
```

Configure no .env:

```
BACKUP_DIR=/var/backups/sistema-clinico-mvp/mysql
BACKUP_RETENTION_DAYS=30
BACKUP_AGE_RECIPIENT=age1SUBSTITUA_PEL0_DESTINATARIO_PUBLICO
```

Prepare o diretório e gere um backup manual para validação:

```
install -d -m 0700 /var/backups/sistema-clinico-mvp/mysql
./scripts/backup_mysql_encrypted.sh
```

O script gera .sql.gz.age e .sha256, nunca persiste o SQL em claro e remove backups com mais de BACKUP_RETENTION_DAYS somente depois de publicar um novo backup válido.

Agende o backup diário depois do teste manual:

```
15 2 * * * cd /opt/sistema-clinico-mvp && /usr/bin/env bash -c 'set -o pipefail;
./scripts/backup_mysql_encrypted.sh 2>&1 | /usr/bin/logger -t sistema-clinico-backup'
```

Para restaurar, copie o backup e o checksum para um ambiente isolado e disponibilize temporariamente a identidade privada fora da VPS de producao:

```
AGE_IDENTITY_FILE=/secure/keys/medsystem-backup.agekey \
./scripts/restore_mysql_encrypted.sh \
/secure/restore-input/sistema_clinico_mysql_YYYYMMDDTHMMSSZ.sql.gz.age
```

O restore exige digitar RESTAURAR:NOME_DO_BANCO conforme o destino exibido. Backup e restore compartilham um lock para impedir execucao concorrente no mesmo projeto. Teste a restauracao trimestralmente em ambiente isolado. Nunca mantenha AGE_IDENTITY_FILE ou a chave privada na VPS de producao.

Retencao e descarte LGPD

Antes de ativar os prazos, obtenha aprovacao do Controlador, DPO, juridico/regulatorio e responsavel assistencial. Simule primeiro:

```
docker compose exec -T backend flask lgpd-retencao --dry-run
```

Depois de revisar as contagens, anote o Hash do plano exibido e confirme um backup valido do mesmo ciclo. A execucao exige tambem chamado aprovado, operador e consulta a preservacoes legais:

```
docker compose exec -T backend flask lgpd-retencao --execute \
--plan-hash HASH_SHA256_DO_DRY_RUN \
--plan-reference REFERENCIA.UTC_DO_DRY_RUN \
--backup-reference sistema_clinico_mysql_YYYYMMDDTHMMSSZ.sql.gz.age \
--approval-reference CHANGE-1234 \
--operator conta-tecnica-responsavel \
--confirm-no-legal-hold
```

O comando aborta se o conjunto destrutivo mudou desde o dry-run. Nesta versao ele exclui somente logs de integracao, filas terminais e auditorias vencidas. Os espelhos SPData e seus vinculos sao monitorados no dry-run, mas permanecem preservados ate homologacao das FKs e concorrencia no MySQL. Prontuario e demais dados clinicos nao fazem parte do descarte automatico. O descarte nao deve ser colocado em cron nesta versao. Consulte a politica completa em docs/politica-retencao-descarte-backup.md e no PDF correspondente.

Diagnostico rapido

Ver status:

```
docker compose ps
```

Ver logs do backend:

```
docker compose logs --tail=200 backend
```

Ver logs do Caddy e emissao do certificado HTTPS:

```
docker compose logs --tail=200 caddy
```

Verificar a revisao atual das migrations, se o backend estiver rodando:

```
docker compose exec backend flask db current
```

Rodar migrations manualmente, apenas para diagnostico ou recuperacao:

```
docker compose run --rm backend flask db upgrade
```

Entrar no container do backend:

```
docker compose exec backend sh
```

Testar conexao Flask internamente:

```
docker compose exec frontend wget -q0- http://backend:5000/
```

Testar MySQL internamente:

```
docker compose exec mysql sh -c 'mysqladmin ping -h 127.0.0.1 -uroot -p"$MYSQL_ROOT_PASSWORD"'
```

Se o build do frontend falhar por falta de memoria, confirme que o frontend/Dockerfile contem `NODE_OPTIONS=--max-old-space-size=4096`. Em VPS muito pequenas, aumente a memoria/swap antes de executar `docker compose up -d --build`.

Observacoes sobre dominio e HTTPS

Este Compose expoe apenas o Caddy nas portas 80 e 443. O Caddy gera e renova automaticamente o certificado HTTPS do dominio definido em `APP_DOMAIN` e encaminha as requisicoes para o container `frontend:3000`.

Antes de subir o deploy, o DNS do dominio precisa apontar para o IP publico da VPS. Se o DNS ainda nao estiver propagado, o Caddy nao conseguira emitir o certificado.

Checklist final

- .env criado na VPS com senhas fortes.
- APP_DOMAIN apontando para o dominio real, sem http:// ou https://.
- Senha da VPS rotacionada caso tenha sido compartilhada.
- VPS - Acesso.md removido do repositório.
- Firebird/SPDATA acessível a partir da VPS.
- Portas 80 e 443 liberadas.
- docker compose up -d --build executado com sucesso.
- Backend ficou healthy, indicando que as migrations automáticas passaram e a API iniciou.
- Aplicação acessível pelo navegador em HTTPS.